

Estimativa estocástica paralela de PI

Nessa atividade foi pedido para fazer a mesma atividade feita anteriormente, mas utilizando as diretivas do openmp para explorar como melhorar uma atividade computacionalmente intensa.

Para ela, foi utilizado uma variável global pi, e variáveis locais pi_ e sign.

A lógica de paralelização consiste em calcular uma parte da sequência por meio do `parallel for` e acumular esse valor em pi_, e ao fim disso utilizar uma operação de soma da variável local pi e pi_ de forma atômica para não gerar uma condição de corrida.

```
#include <omp.h>
#include <stdio.h>

long long iters = 100000000000;

int par_exec() {
    double pi = 0;

    double inicio, fim;

    inicio = omp_get_wtime();

    // 4 é a quantidade de cores que eu tenho no Laptop
    omp_set_num_threads(4);
    #pragma omp parallel shared(pi)
    {
        double pi_ = 0;
        double sign = 1;
        // Só usando o omp parallel for dá uma condição de corrida desgraçada, o
        // valor fica todo quebrado.
        #pragma omp for
        for (size_t i = 1; i < iters; i += 2) {
            pi_ += (1 / (float)i) * sign;
            sign = -sign;
        }

        #pragma omp atomic
        pi += pi_;
    }

    fim = omp_get_wtime();

    pi = 4 * pi;

    printf("-----\n");
    printf("PARALLEL EXEC\n");
    printf("Iters: %lld ", iters);
    printf("Elapsed wall time: %f seconds\n", fim - inicio);
    printf("Aprox: %f\n", pi);
    printf("-----\n");
```

```

    return 0;
}

int seq_exec() {
    double pi = 0;
    double inicio, fim;

    inicio = omp_get_wtime();

    {
        double pi_ = 0;
        double sign = 1;
        for (size_t i = 1; i < iters; i += 2) {
            pi_ += (1 / (float)i) * sign;
            sign = -sign;
        }

        pi += pi_;
    }

    fim = omp_get_wtime();

    pi = 4 * pi;

    printf("-----\n");
    printf("SEQ EXEC\n");
    printf("Iters: %lld ", iters);
    printf("Elapsed wall time: %f seconds\n", fim - inicio);
    printf("Aprox: %f\n", pi);
    printf("-----\n");

    return 0;
}

int main() {
    seq_exec();
    par_exec();
    return 0;
}

// > clang -fopenmp tasks/task5.c -O3
// > ./a.out
// -----
// SEQ EXEC
// Iters: 100000000000 Elapsed wall time: 51.507647 seconds
// Aprox: 3.141593
// -----
// -----
// PARALLEL EXEC
// Iters: 100000000000 Elapsed wall time: 21.810869 seconds
// Aprox: 3.141593
// -----

```